

AULA 7

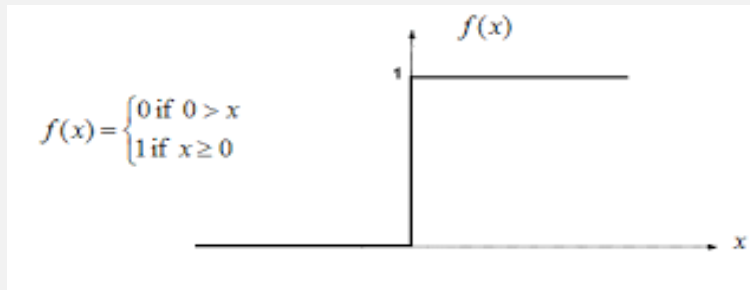
Introdução a controle

O QUÊ É CONTROLE?

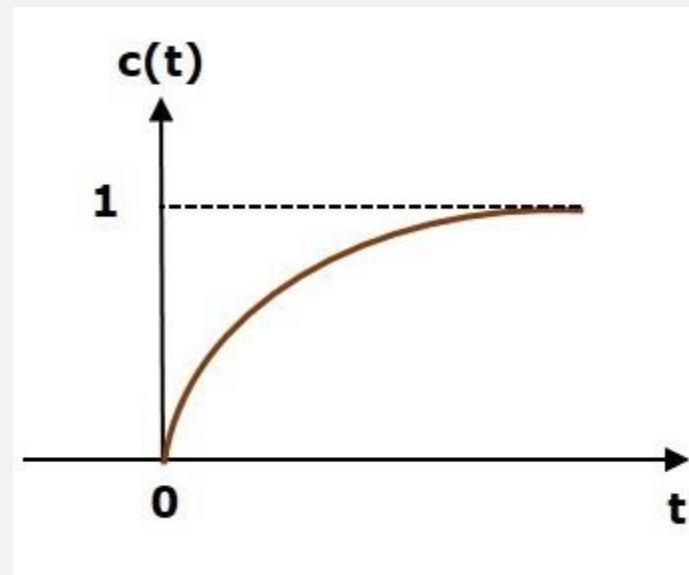
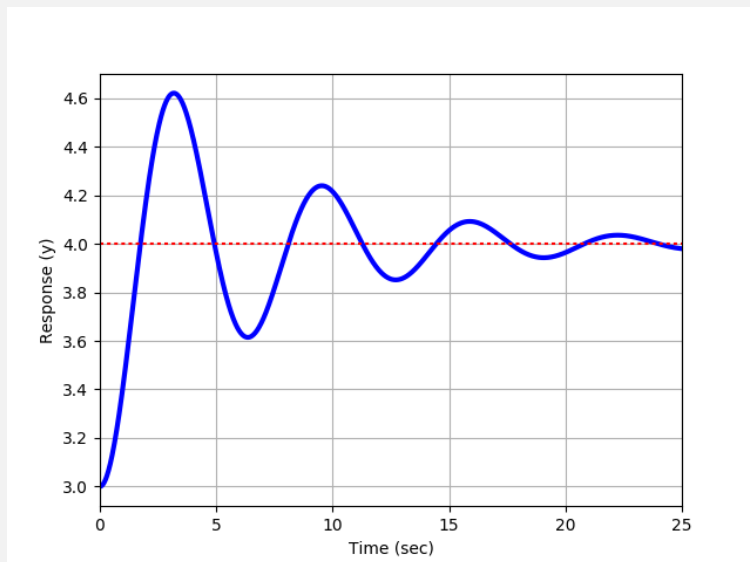
- O que NÃO é controlar *algo*?
 - Ligar ou desligar
 - Determinar/comandar o objetivo
 - Usar um controle para mudar o canal
- O que é então?
 - Controle é **conduzir** a variável que se deseja controlar para um ponto patamar com um **comportamento pré-determinado**, minimizando efeitos de perturbações através de um **conjunto de sinais de controle**.
 - Controle se trata tanto do **como** quanto do **onde**

O QUÊ É CONTROLE?

- Exemplo de comportamento : Temperatura de um forno



Como vocês gostariam que fosse a resposta do seu forno?



O QUÊ É CONTROLE?

- Exemplo de especificação para um controlador:
 - Mantenha a velocidade de um carro em 80 km/h, chegue nessa velocidade em até 30s sem nunca passar de 82 km/h
- Perceba que um controlador (um motorista nesse caso) não apenas deve fazer o carro ter certo resultado final, mas também como ele deve obter esse resultado

TIPOS DE CONTROLE

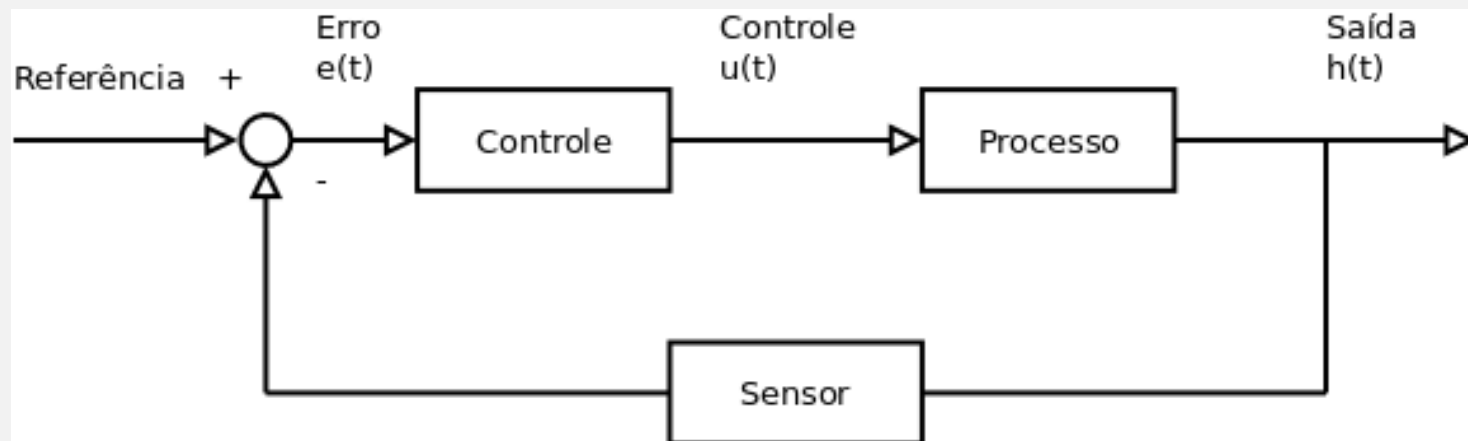
- Malha aberta:
 - Nesta abordagem não há qualquer feedback sobre o andamento do processo para determinar o sinal de controle a ser aplicado. O sinal de controle é aplicado sem que seja feita a medição do sinal de saída, não há qualquer adaptação ao processo.
 - Uma máquina de lavar roupa
 - Um motorista cego, surdo e sem percepção de aceleração



Perfume de Mulher (1992)

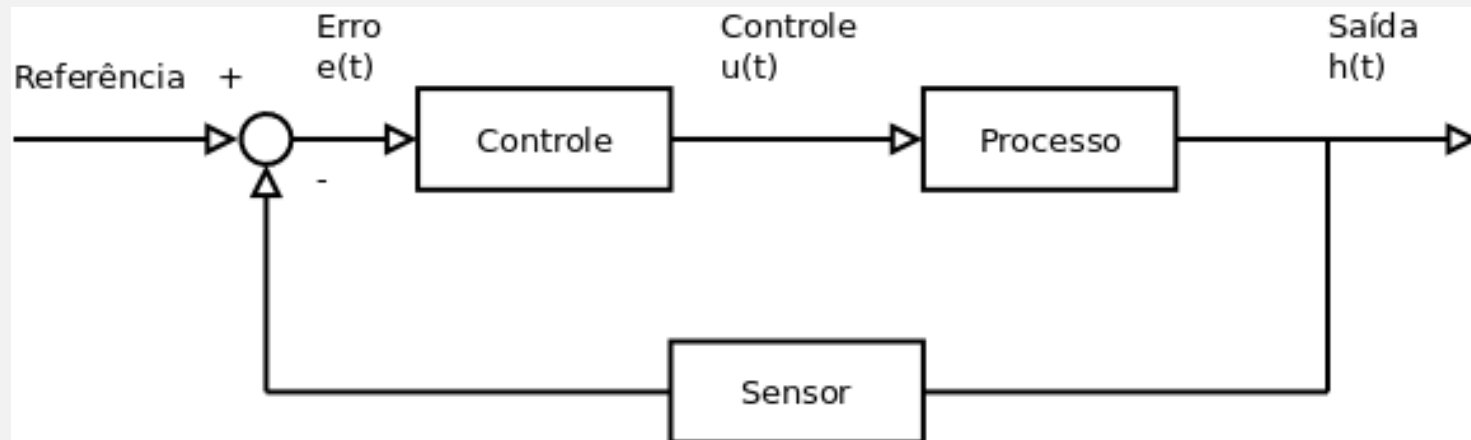
TIPOS DE CONTROLE

- Malha Fechada:
 - Neste método informações sobre como **a saída** está evoluindo são utilizadas para determinar o sinal de controle que deve ser aplicado ao processo a cada instante. A medição é feita a partir de um sensor (ou transdutor).
 - O sinal de saída é comparado com o sinal de referência (o desejado), a diferença entre os dois sinais é chamado de **erro**.
 - A comparação entre a saída e a referência é chamada de realimentação. A entidade que utiliza do sinal de erro é chamado de **controlador**

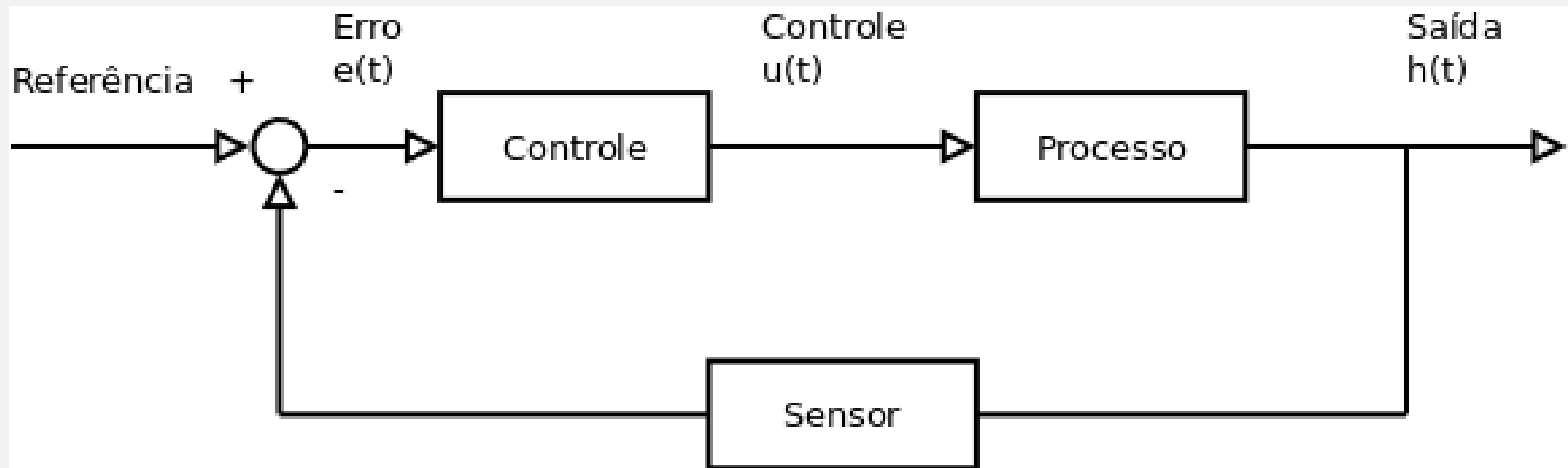


TIPOS DE CONTROLE

- Malha Fechada:
 - Exemplos:
 - Um forno com sensor de temperatura
 - Um motorista normal
 - Um robô*



O QUÊ É CONTROLE?

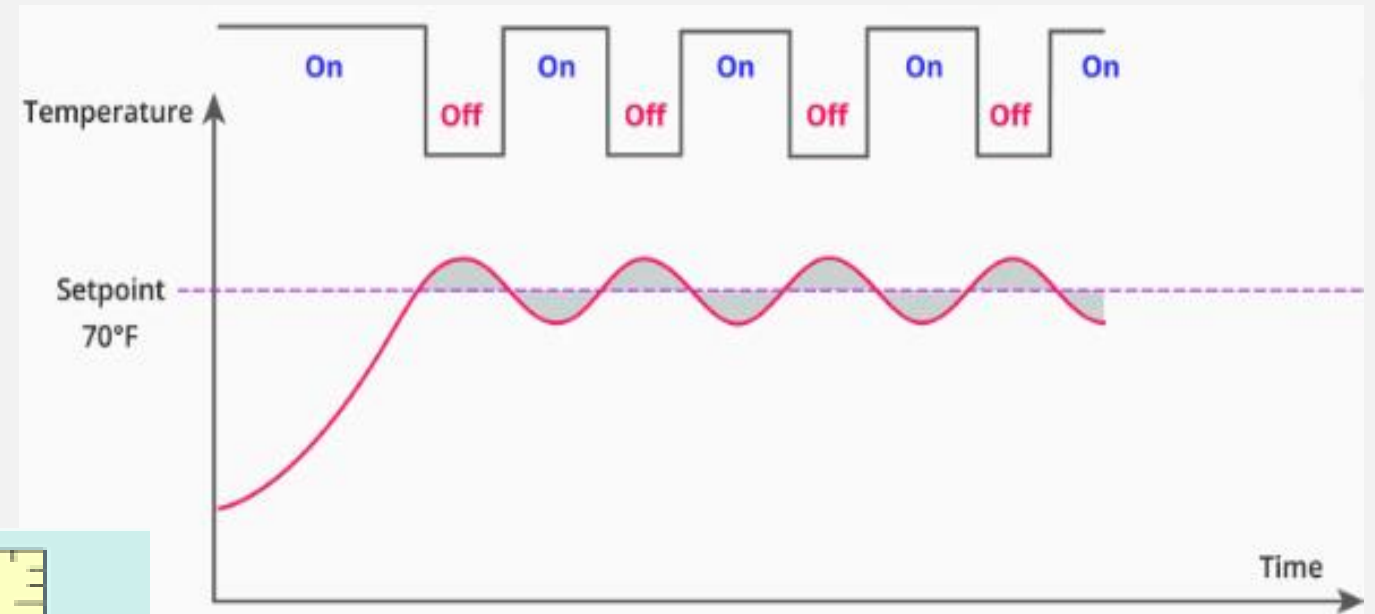
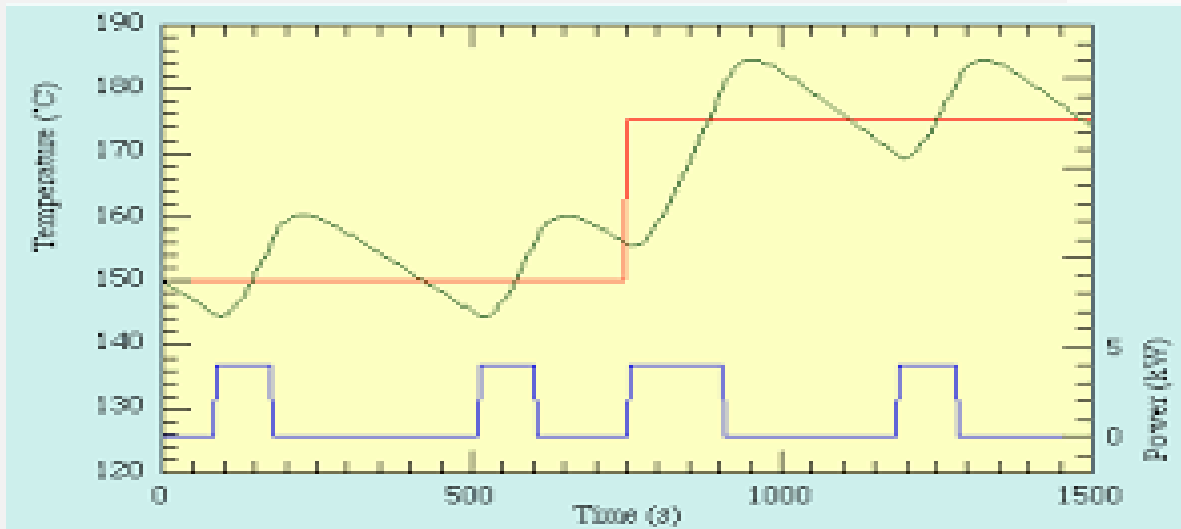


AÇÕES DE CONTROLADOR

- Como o controlador utiliza do erro para controlar?
 - Vamos explorar 4 tipos de ações
 - Bang-bang
 - **P**roportional
 - **I**ntegral
 - **D**erivativa

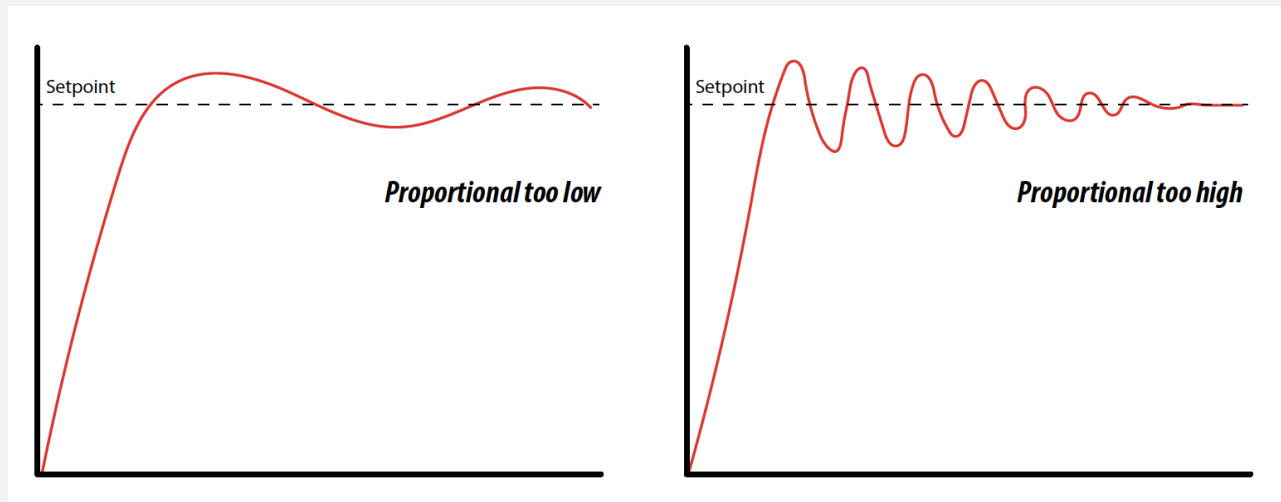
AÇÕES DE CONTROLADOR

- **Bang-Bang** : Basicamente um comparador
 - Se o erro for positivo (saída < referência) um sinal de controle é aplicado, caso contrário outro sinal é aplicado



AÇÕES DE CONTROLADOR

- **Proporcional** : Aplicamos um sinal de controle proporcional a magnitude do erro
 - Intuição : Quanto maior o erro, maior deve ser o sinal de controle para se aproximar mais rapidamente da referência desejada
 - Para esse controlador, o projetista determina o K (ganho proporcional) $u_p(t) = Ke(t)$
 - Quando maior o K maior a resposta do sistema ao erro
 - Um ganho muito grande tende a levar sistemas a comportamentos oscilatórios e até instabilidade

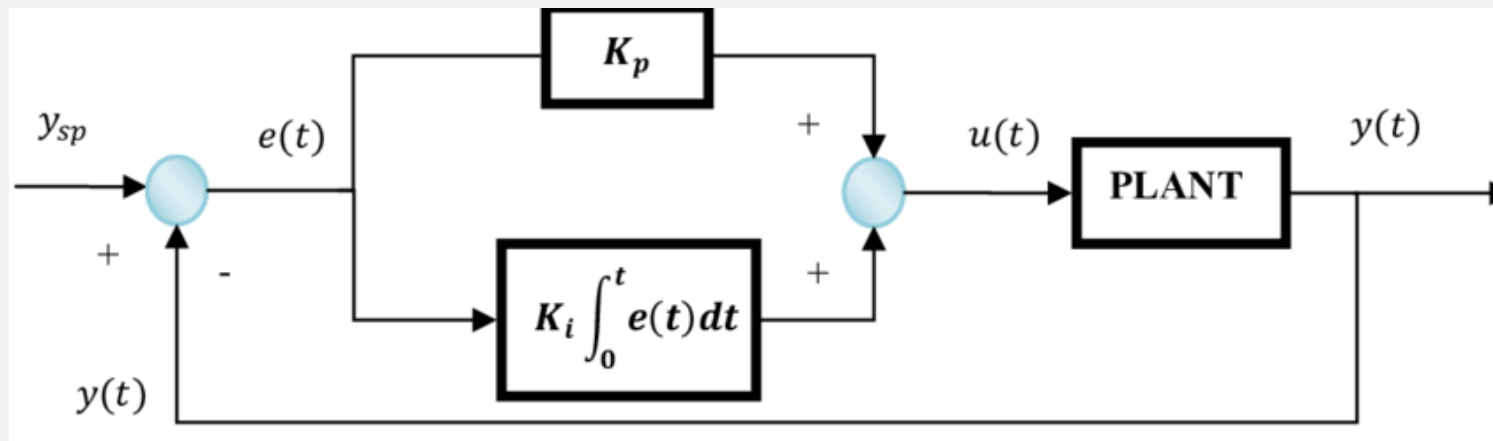


Note que quando o erro é nulo a ação proporcional também é nula, ou seja, essa ação sozinha não consegue manter um sistema na referência, apenas leva-lo* até lá

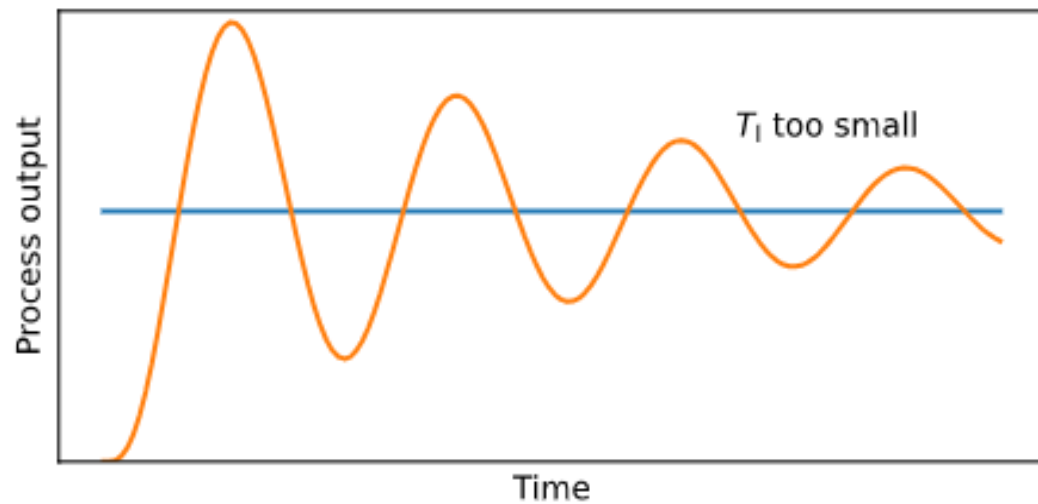
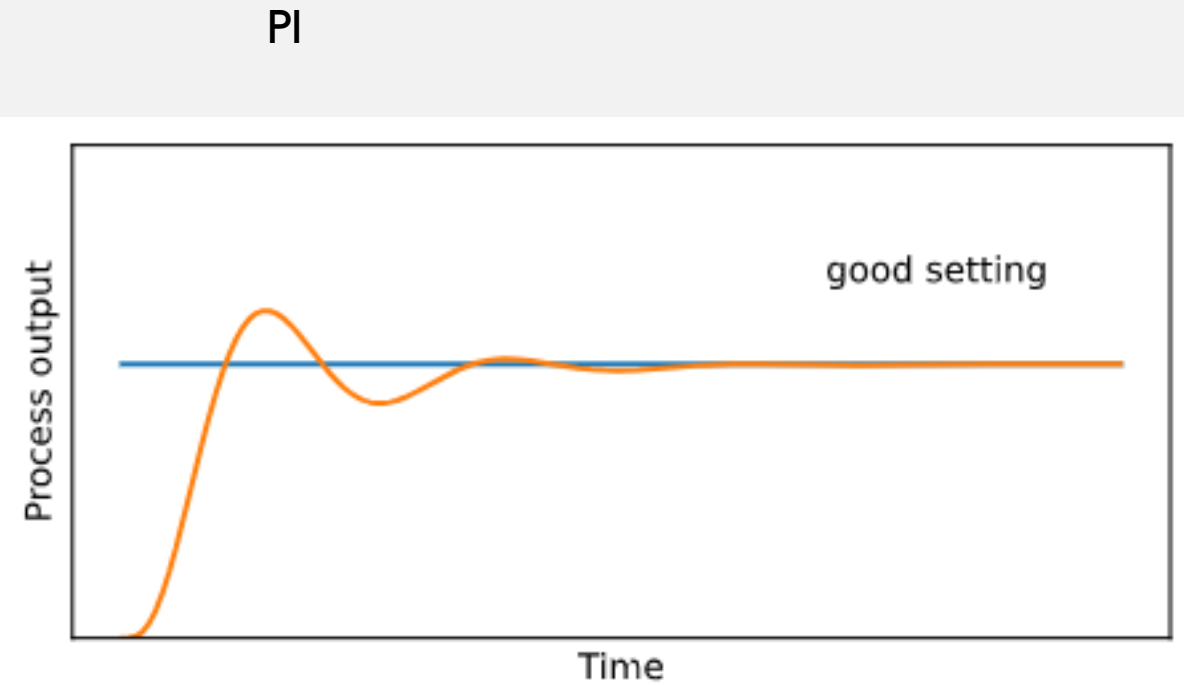
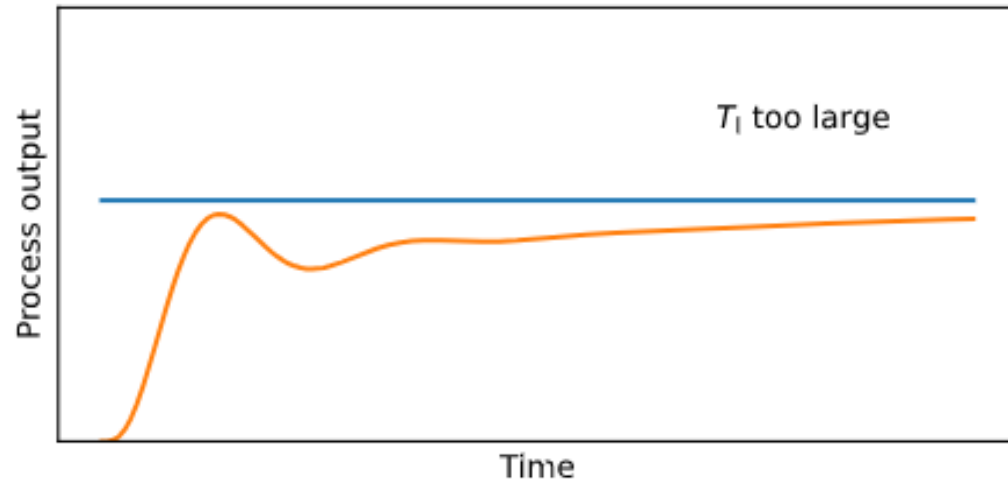
AÇÕES DE CONTROLADOR

- **Integral:** Aplicamos um sinal de controle proporcional a integral(soma) do erro até o momento
 - Intuição : Como uma memória ou armazenamento de energia. Somamos o erro ao longo do tempo, "lembrando" o que é preciso fazer para chegar na referência.
 - Para esse controlador, o projetista determina o $1/T_i$ (ganho integral)
 - Essa ação, apesar de ser capaz de manter sistemas em determinadas referências, tende a deixar a resposta do sistema lenta e torna-lo instável quando usada isoladamente.

$$u_i(t) = \frac{1}{T_i} \int_0^t e(\tau) d\tau$$



AÇÕES DE CONTROLADOR



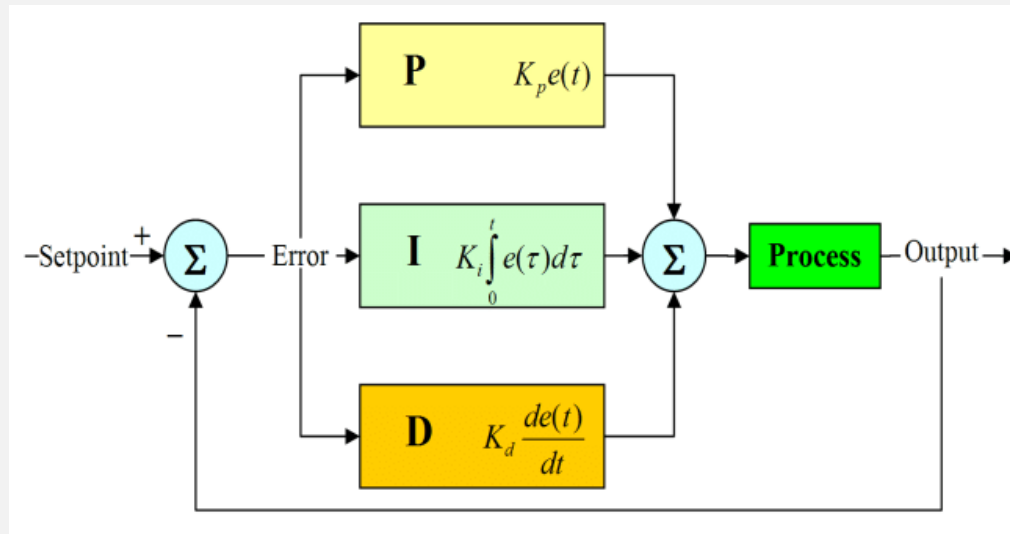
AÇÕES DE CONTROLADOR

- **Derivativo:** Aplicamos um sinal de controle proporcional a derivada (taxa de variação) do erro
 - Intuição : Uma ação preditiva, reage a como o erro está mudando, não ao seu valor em si. Se nosso erro está caindo rapidamente (derivada < 0), podemos diminuir nosso sinal de controle
 - Para esse controlador, o projetista determina o T_d (ganho derivativo)
 - Essa ação por si só não é capaz de levar a saída para lugar nenhum, mas quando combinado com as demais é capaz de acelerar a resposta do sistema

$$u_d(t) = T_d \frac{d}{dt} e(t)$$

AÇÕES DE CONTROLADOR

- Como percebido, as ações não são suficientes sozinhas, por isso combinamos pares ou as três para obter comportamentos desejados
 - PI : Controlador Proporcional Integral
 - PD : Controlador Proporcional Derivativo
 - PID : Controlador Proporcional Integral Derivativo

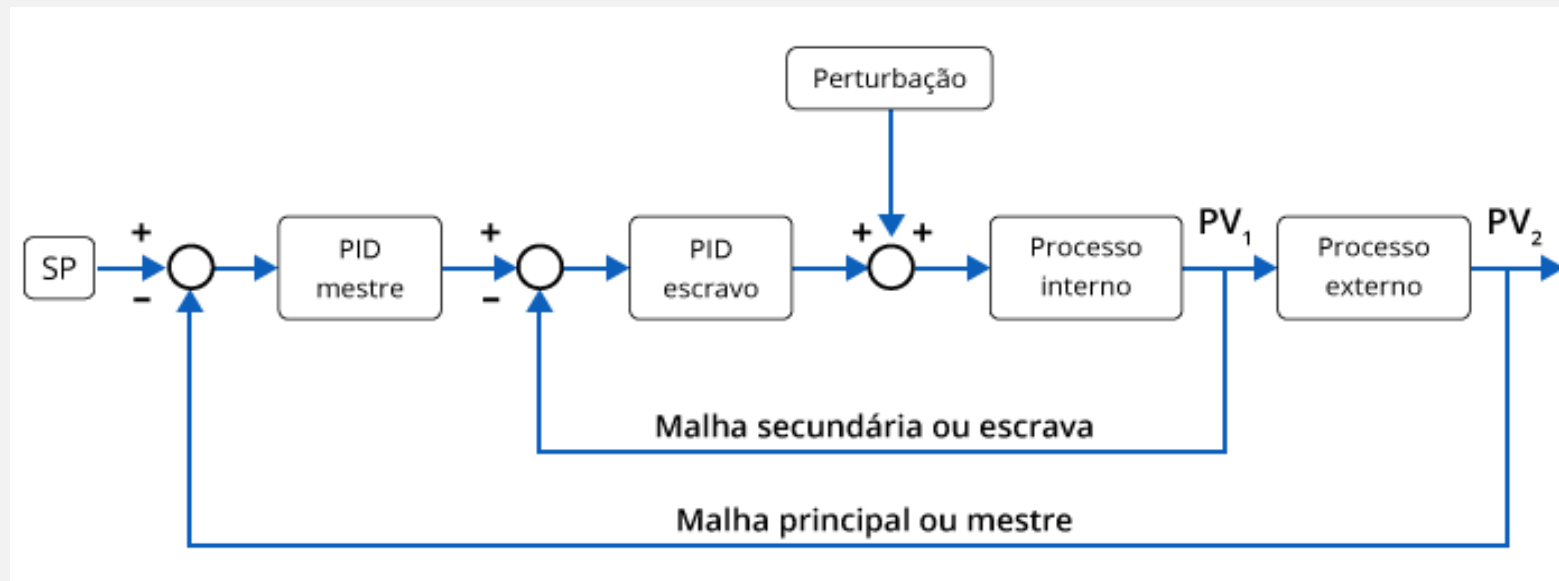


Exemplo



E O QUE CONTROLAMOS?

- Quase tudo, mas não diretamente
- Não tem como comandar **diretamente** a posição de um robô, por exemplo
 - Será uma série de malhas de controle (posição -> velocidade -> aceleração) que controlam do ponto de vista da malha externa, a posição

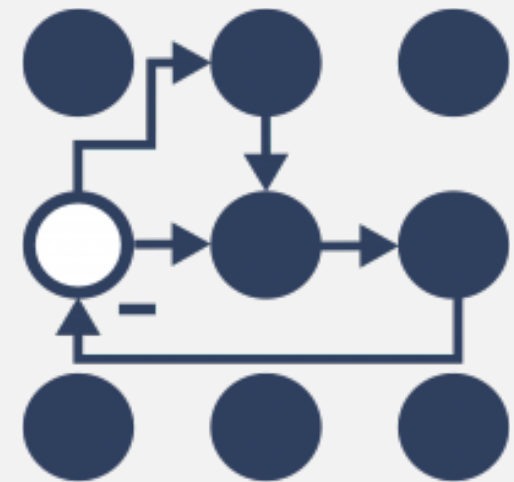


ROS2_CONTROL

Controle dentro do ROS2

COMO CONTROLAR UM ROBÔ COM O ROS?

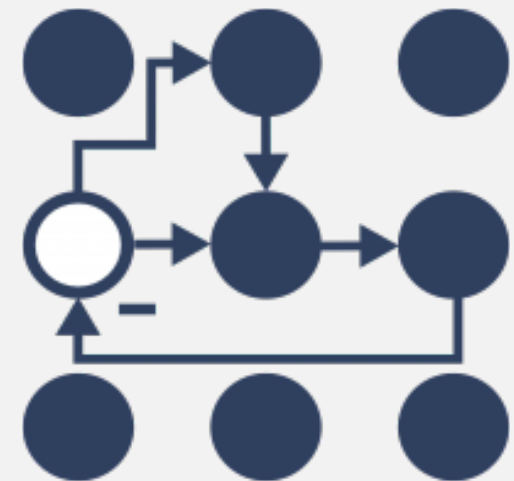
- Digamos que você tem um robô, e você gostaria de usar bibliotecas específicas do ROS 2 (como MoveIt! ou Nav2) com ele.
 - Ninguém quer reescrever elas para se comunicar diretamente com o hardware do robô
- Idealmente gostaríamos de um processo de desenvolvimento modular, que separasse as duas partes:
 - Escrever drivers para componentes dos robôs (e.g. sensores, rodas, etc.)
 - Escrever controladores que dizem o que fazer com esses componentes (ler dados, mover rodas)
 - ...e fazer eles falarem uma linguagem comum (posso trocar um controlador por outro sem muitos problemas)
- É isso que é o objetivo do `ros2_control`!



Logotipo do `ros_control`

○ SISTEMA ROS2_CONTROL

- O desenvolvimento de controladores no ROS(2) é geralmente feita com o *framework* `ros(2)_control`. Ele implementa:
 - Camada de abstração de hardware e controle de baixo nível
 - Capacidades de gerenciar e desenvolver controladores, que são laços de realimentação para seguimento de referências
 - Ferramentas *real-time safe* para rodar controladores
 - Composição/encadeamento de controladores (conectar um em série a outro – compartmentalizar funções de cada controlador)
- Documentação do projeto (com tutoriais): <https://control.ros.org/jazzy/index.html>
- Repositórios: <https://github.com/ros-controls>



Logotipo do ros_control

PORQUE USAR O SISTEMA ROS2_CONTROL: DESEMPENHO EM TEMPO REAL

Ok, mas por que não usamos só comunicação via tópicos?

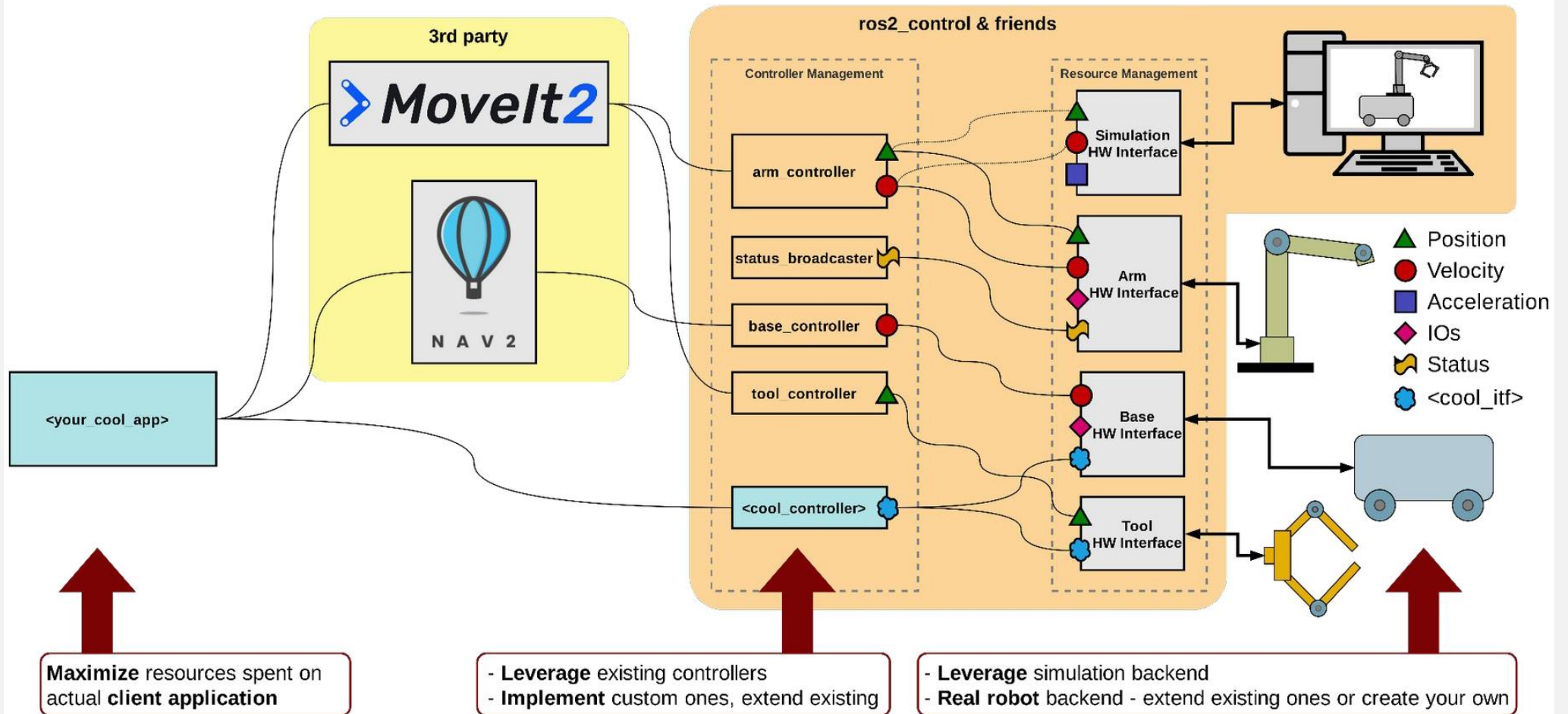
- Controle de sistemas físicos geralmente precisa operar em uma frequência muito maior que a do sistema
 - Não adianta ter só o código em simulação funcionando
 - Tipicamente a implementação de sistemas de controle é feita em microcontroladores, CLPs, e outras plataformas que **garantem** a execução em **tempo real**
- A maior parte dos sistemas operacionais do nosso dia-a-dia (Windows, Mac) não são e não têm como serem *real-time safe*, nem na média.
- Linux é uma exceção: apesar de não ter como ser um verdadeiro RTO tem algumas soluções:
 - PT_PREEMPT: patches ao kernel do Linux permitem tarefas de tempo real. Integrado completamente no kernel v6.12
 - Xenomai: um segundo kernel verdadeiramente RTOS operando concomitantemente com o kernel do Linux
- `ros2_control` permite usar kernels e operação em tempo real

À título de curiosidade...

Alto desempenho não necessariamente significa tempo real, e vice-versa! Um sistema de tempo real é aquele no qual se pode garantir que um dado processamento será realizado entre dois instantes de tempo, não antes e nem depois. São tradicionalmente distinguidos três tipos de sistemas de tempo real [W. F. Lages, *Sistemas de Tempo Real*, 2014]:

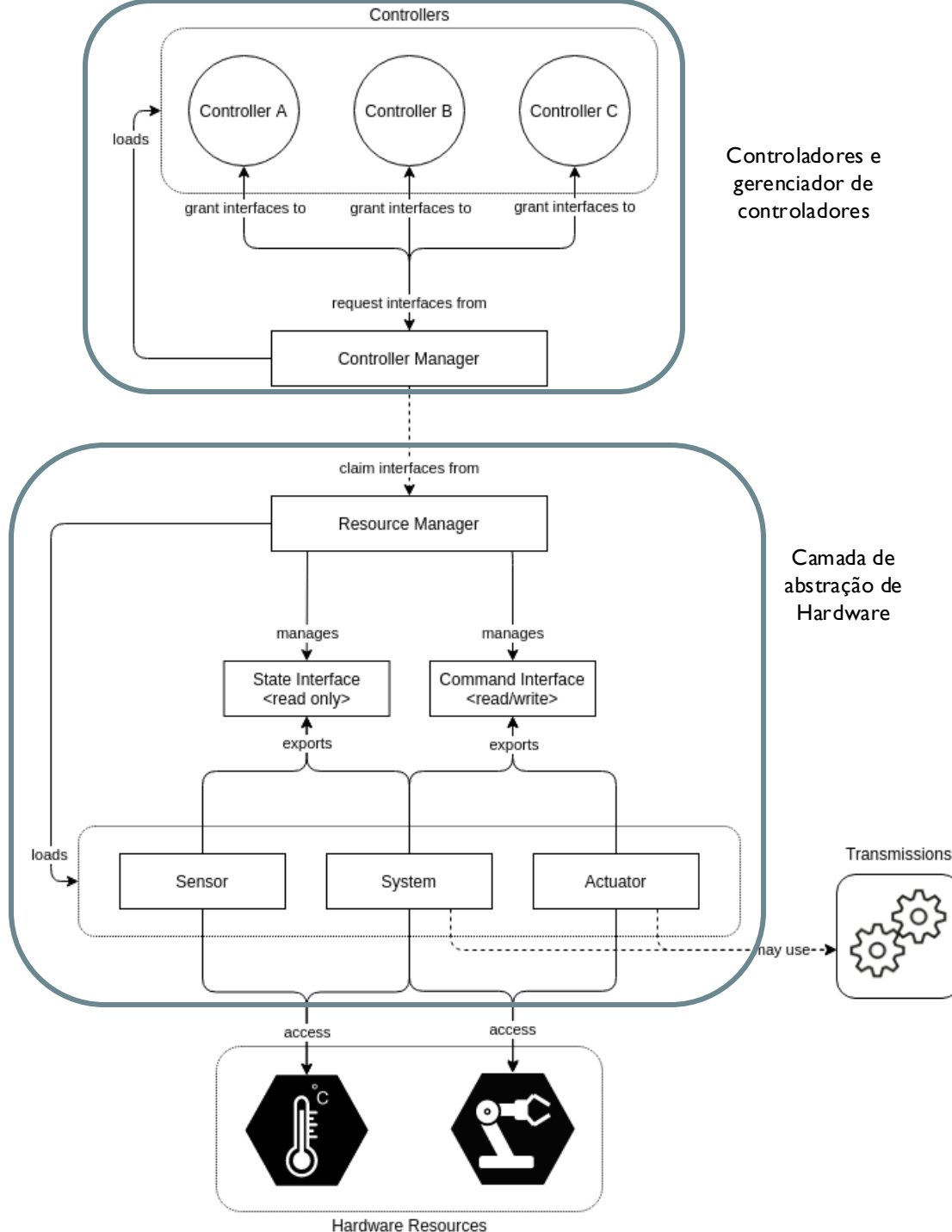
- 1) *Hard real-time*: Violação de restrições temporais, mesmo que uma única vez, é uma falha no sistema.
- 2) *Firm real-time*: Violação de restrições temporais implica em degradação de performance, mesmo se toleradas.
- 3) *Soft real-time*: Violação de restrições temporais pode ocorrer esporadicamente – as restrições devem ser satisfeitas apenas em um sentido médio.

○ SISTEMA ROS2_CONTROL



ARQUITETURA DO ROS2_CONTROL

205

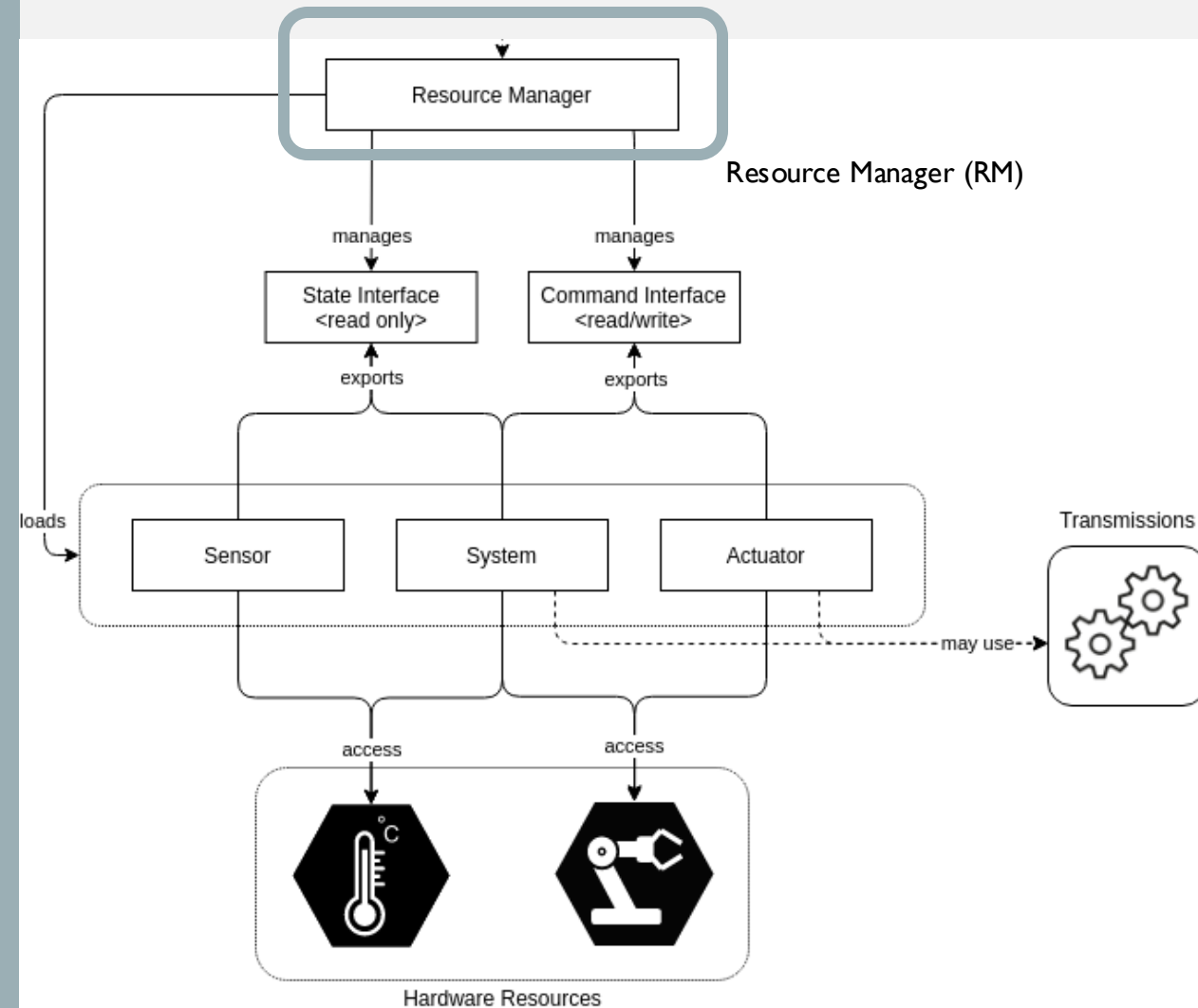


- Dividido em duas "camadas":
 - Controladores (gerenciados pelo Controller Manager)
 - Camada de abstração de Hardware: interfaces de hardware, administradas pelo Resource Manager
- Cada um é responsável por administrar um conjunto de plugins:
 - Controllers: derivados da classe ControllerInterface. Solicitam interfaces ao Controller Manager
 - HardwareInterface: Podem ser de 3 subtipos, Actuator, System ou Sensor. Tipo diz como ele interage com um Controller.
 - Actuator: hardware de IGD, e.g. motores, válvulas. Tipicamente apenas escritos.
 - Sensor: hardware usado para medir o ambiente. Apenas pode ser lido.
 - System: hardware complexo, multi-GDL. Usado quando se tem um só canal de comunicação, e.g. robôs industriais. Pode ser lido e escrito.

O SISTEMA ROS2_CONTROL: RESOURCE MANAGER (RM)

Classe que abstrai o hardware físico e seus drivers para o `ros2_control`. Ele lê o URDF do robô e instancia os Recursos de Hardware e componentes, que são carregados como plugins (via [pluginlib](#), a forma padrão de carregar plugins no ROS2) .

Ele gerencia o ciclo de vida dos componentes (i.e. o que cada componente faz quando é carregado, descarregado, ou interfaceia com um controlador) e a interface dos componentes via `state_interface` ou `command_interfaces`.



O SISTEMA ROS2_CONTROL: RECURSOS DE HARDWARE (HR)

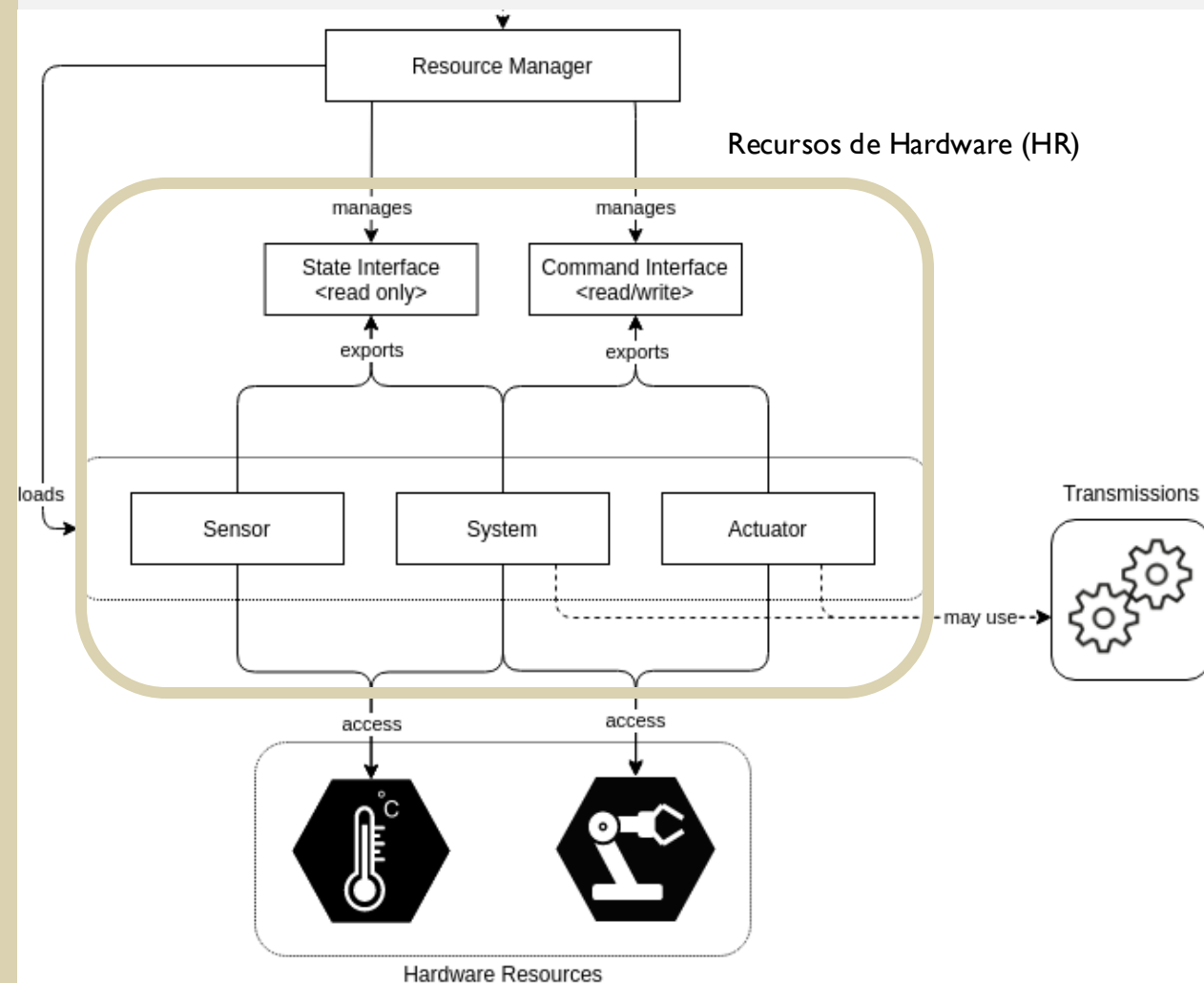
Fazem a comunicação com o hardware físico (ou simulado), e são exportados como plugins.

Cada HR é carregado dinamicamente no *runtime*.

O código do plugin diz como ele se comporta na interface com o `ros2_control`.

Se não tivermos o plugin de interface de hardware de um dado componente, ou implementamos ele, ou importamos ele (via fabricante, repositórios git, etc...)

A configuração de um HR no `ros2_control` é realizada via tags no URDF do robô.



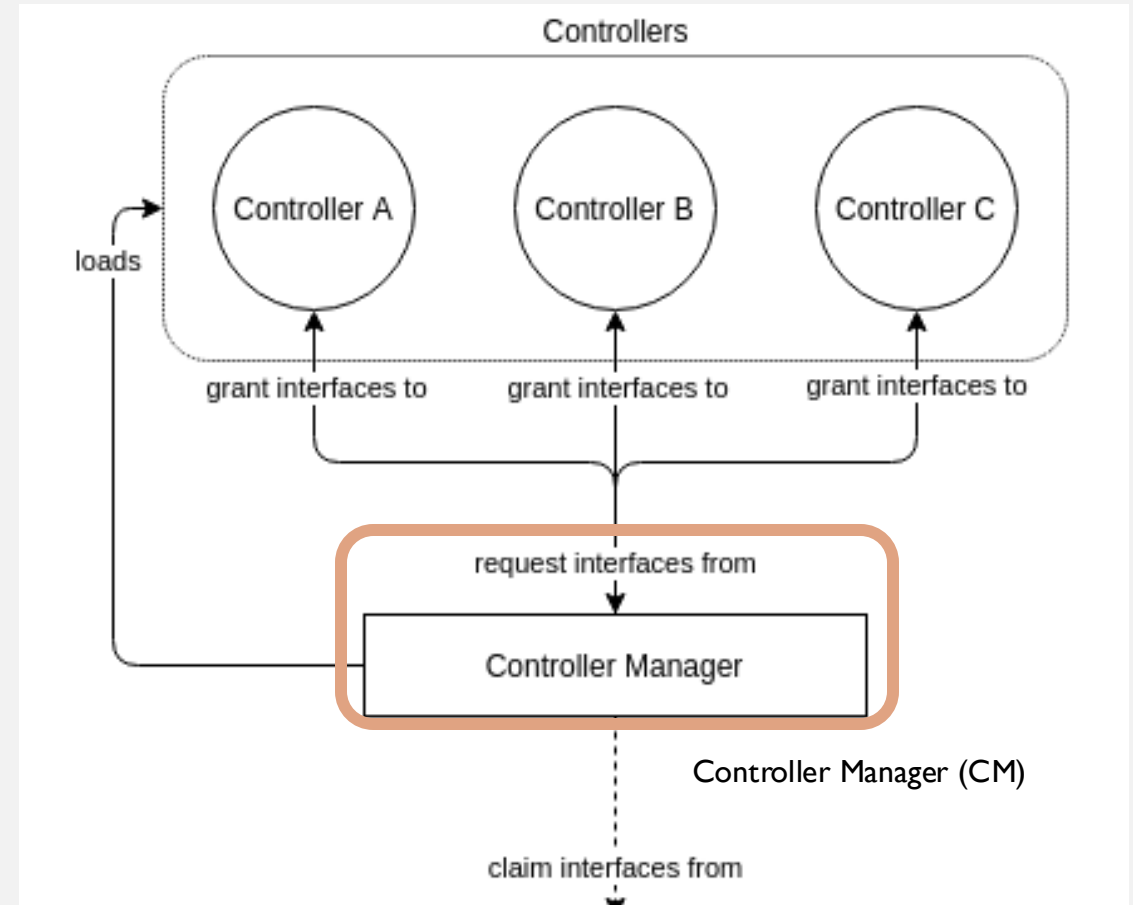
O SISTEMA ROS2_CONTROL: CONTROLLER MANAGER (CM)

Nó (`rclcpp::Node`) que conecta os controladores e o lado de abstração de hardware no framework.

É o **principal componente** do framework e o "ponto de entrada" para usuários - interagimos com ele via serviços.

O CM gerencia os controladores e as interfaces que estes requerem. Ele conecta os controladores com os HRs através do RM.

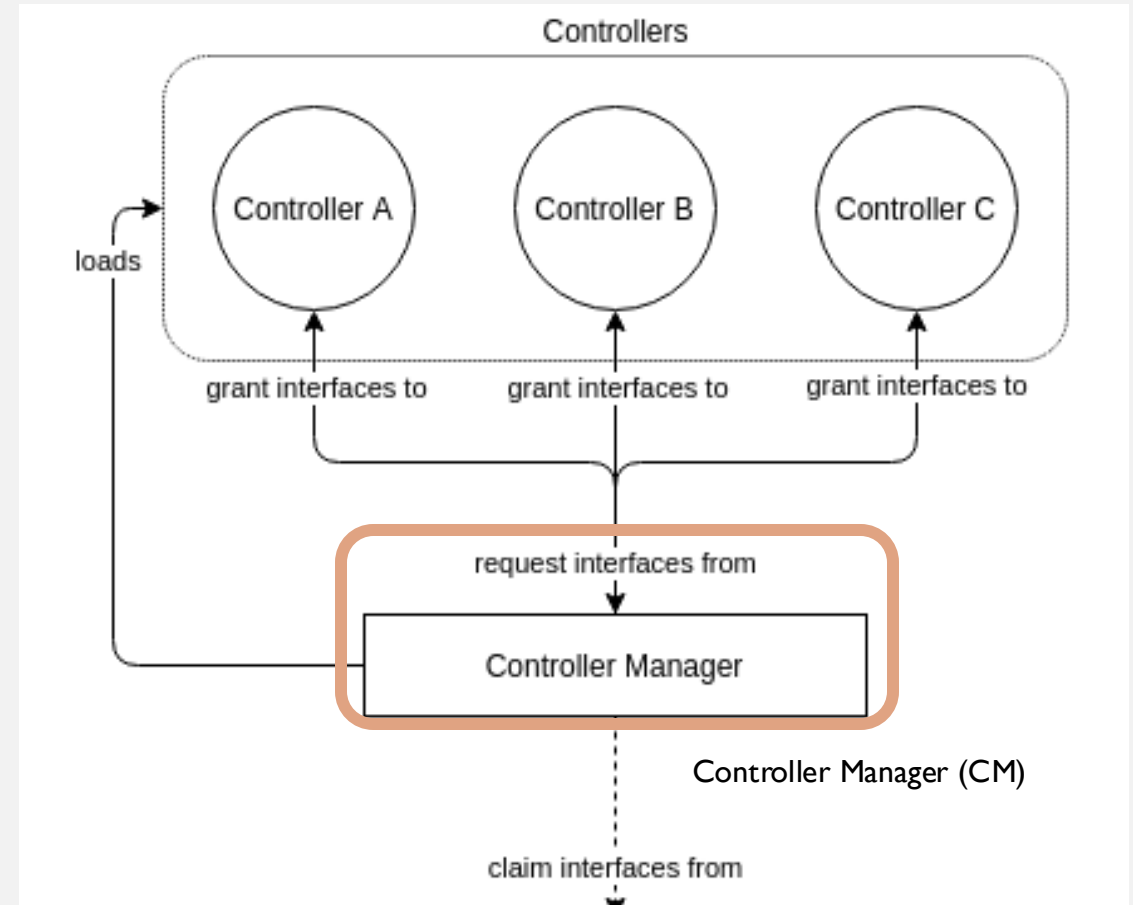
É responsável por permitir o acesso dos controladores ao hardware via as interfaces de comando e estado disponibilizadas.



O SISTEMA ROS2_CONTROL: CONTROLLER MANAGER (CM)

Método do Controller Manager:

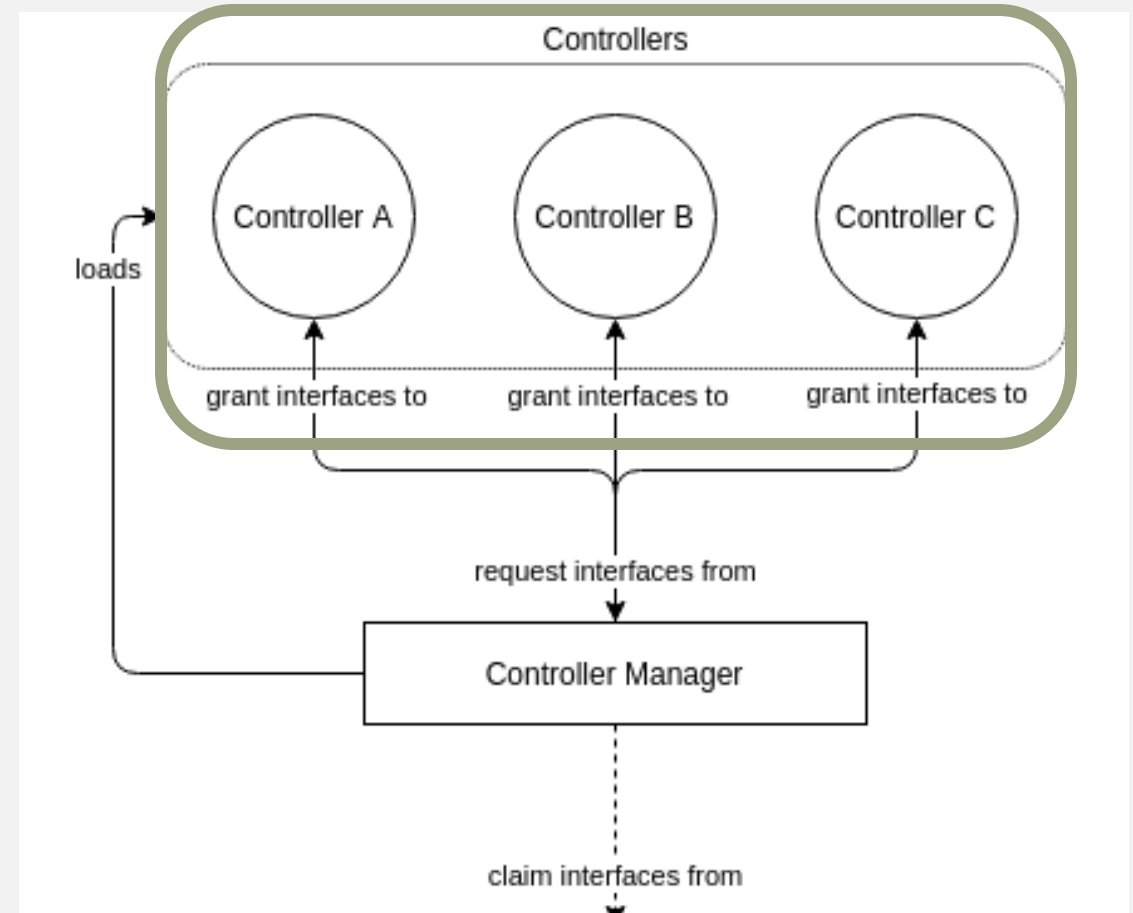
- `read()`:
 - Lê dados do hardware e atualiza interfaces
 - Lida com erros de leitura
- `update()`:
 - Roda o laço de atualização do estado dos controladores
 - Gerencia troca de controladores
 - Mantém a taxa de atualização
- `write()`:
 - Escreve dados ao hardware vindo das interfaces atualizadas pelo controlador
 - Lida com erros de escrita



O SISTEMA ROS2_CONTROL: CONTROLADORES

Bibliotecas carregadas dinamicamente pelo CM como plugins, que usam a interface da classe `ControllerInterface`. São configurados usando arquivos `.YAML`.

Nem todo componente que tem interface `ControllerInterface` do `ros2_control` é um controlador de fato; alguns realmente são, outros são broadcasters (publicam dados de sensores), e ainda outros fazem outras coisas (e.g. filtros).

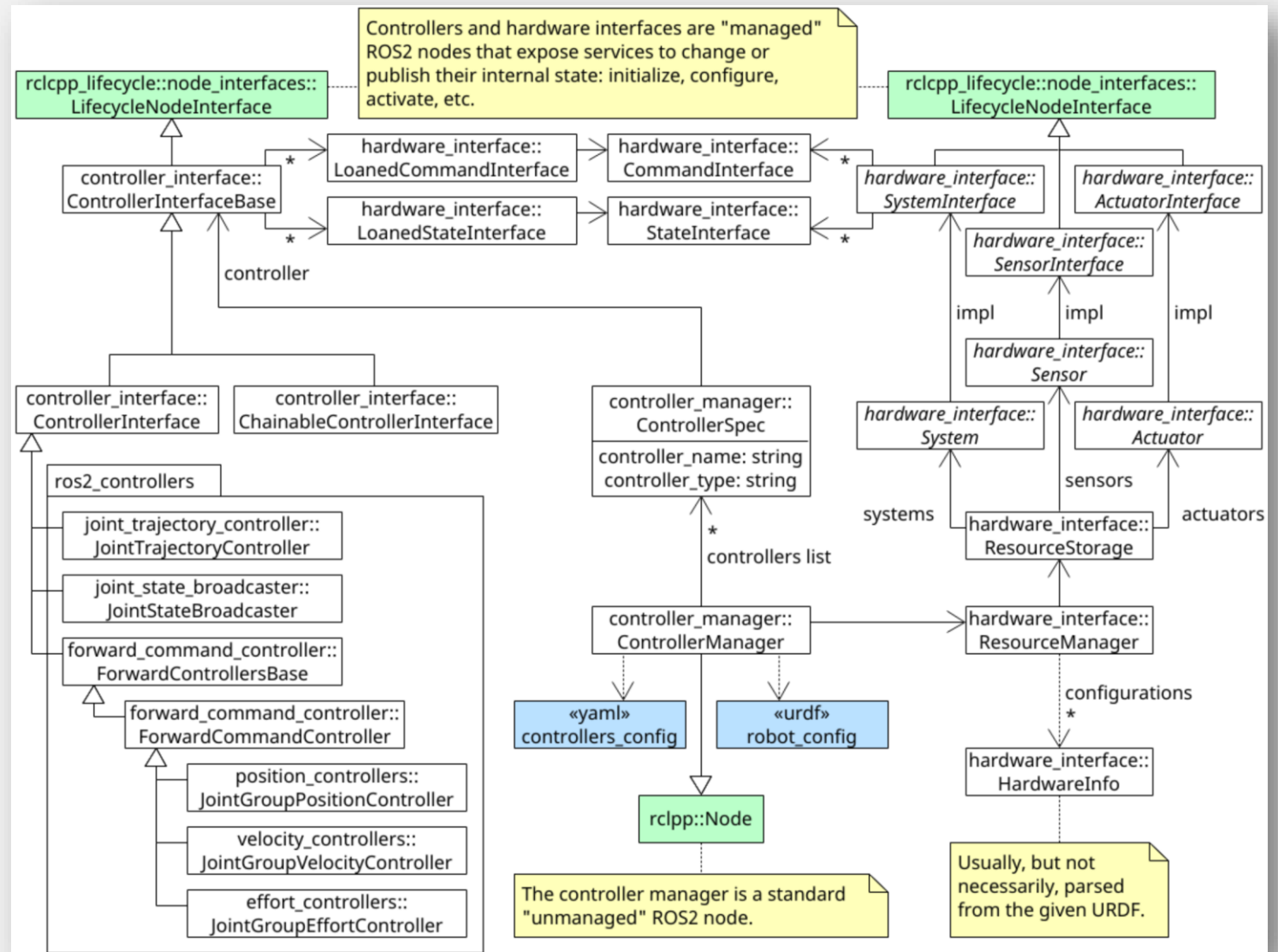


IMPLEMENTAÇÃO DO ROS2_CONTROL

Nós que implementam as subclasses ControllerInterface e HardwareInterface são "[nós administrados](#)" (lifecycle nodes) do ROS2.

Eles são diferentes de nós normais – eles expõem diferentes serviços para publicar ou alterar seu estado interno durante a sua inicialização, configuração, ativação, etc.

Diagrama de classes em UML do sistema ros2_control



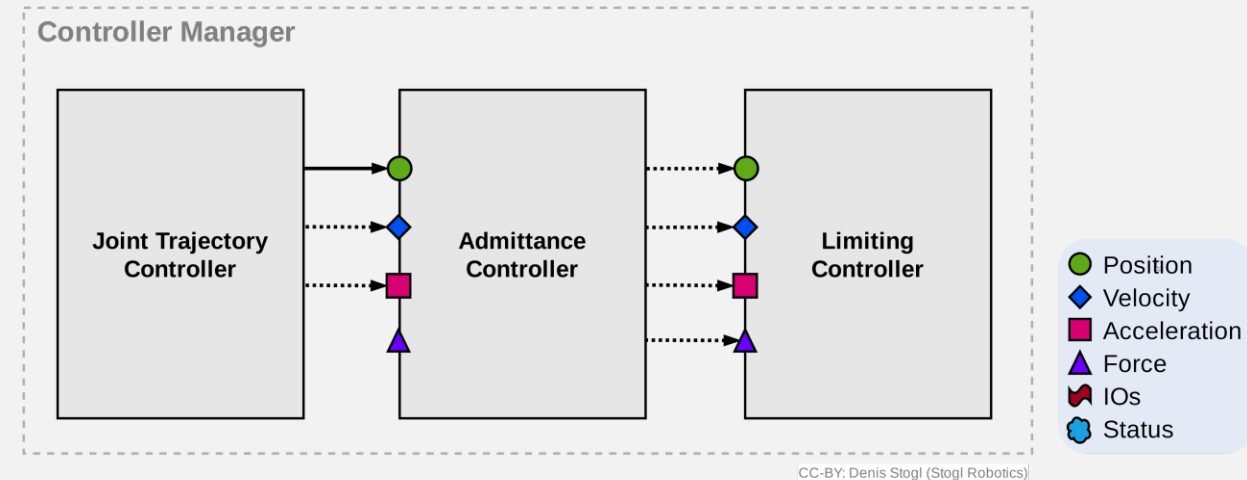
IMPLEMENTAÇÃO DO ROS2_CONTROL

Classe ControllerInterface:

- Instâncias comunicam com hardware através de interfaces de comando e status
- Comunicam com ROS 2 via tópicos
- Podem atualizar e forma síncrona ou assíncrona, com diferentes taxas de atualização

Classe ChainableControllerInterface:

- Extensão de ControllerInterface
- Permitem cascatear controladores
 - Um controlador gera a referência para o outro seguir
 - Permite melhorar o desempenho do sistema de controle (se bem projetado)
 - No `ros2_control`, podemos colocar um controlador que age como um filtro para o outro



Cascadeamento de controles no `ros2_control`

COLOCANDO UMA INTERFACE NO ROBÔ

- Primeira coisa: precisamos dizer o que tem a ser controlado no nosso robô
- Feito no URDF: grupo de tags `<ros2_control>`
 - Pode ser de tipo "sensor", "actuator" ou "system"
 - Especifica as interfaces disponíveis de estado e comando do robô.
- Tags `<hardware>` e `<plugin>` dizem ao `ros2_control` para carregar um `HardwareInterface`
- Tags `<joint>` especificam quais interfaces o plugin carregado vai oferecer que estiverem associadas a uma das juntas do robô

```
<!-- Exemplo de inserção de tags ros2_control no urdf: RRBot -->
<ros2_control name="RRBotSystemMultipleGPIOs" type="system">
  <hardware>
    <plugin>ros2_control_demo_hardware/RRBotSystemPositionOnlyHardware</plugin>
    <param name="example_param_hw_start_duration_sec">2.0</param>
    <param name="example_param_hw_stop_duration_sec">3.0</param>
    <param name="example_param_hw_slowdown">2.0</param>
  </hardware>
  <!-- Grupo de tags de junta - especificam interfaces (comando e estado) -->
  <joint name="joint1">
    <command_interface name="position">
      <param name="min">-1</param>
      <param name="max">1</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>
  <joint name="joint2">
    <command_interface name="position">
      <param name="min">-1</param>
      <param name="max">1</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>

  ... (continua)
```

COLOCANDO UMA INTERFACE NO ROBÔ

- Tags `<gpio>` são como as tags `<joint>`, mas descrevem portas de entrada e saída genéricas (digitais ou analógicas) do robô que não possam ser associadas a uma junta
- Também possuem tags `<command>` ou `<state>` para especificar o tipo de interface
- Ex: válvula pneumática, sensor de temperatura

```
<!-- Continuação do exemplo do slide anterior -->
<gpio name="flange_digital_I0s">
  <!-- Interfaces de comando -->
  <command_interface name="digital_output1"/>
  <command_interface name="digital_output2"/>
  <!-- Interfaces de estado -->
  <state_interface name="digital_output1"/>
  <state_interface name="digital_input1"/>
  <state_interface name="digital_input2"/>
</gpio>

<!-- Comunicação pode ser por canais analógicos -->
<gpio name="flange_analog_I0s">
  <command_interface name="analog_output1"/>
  <state_interface name="analog_output1">
    <!-- Interfaces também podem ter parâmetros -->
    <param name="initial_value">3.1</param>
  </state_interface>
  <state_interface name="analog_input1"/>
  <state_interface name="analog_input2"/>
</gpio>

<gpio name="flange_vacuum">
  <command_interface name="vacuum"/>
  <state_interface name="vacuum"/>
</gpio>
</ros2_control>
```

COLOCANDO UMA INTERFACE NO ROBÔ

- Tags <sensor> servem para agrupar múltiplas state interfaces que descrevem e.g. estado interno do hardware
 - Ex ao lado: sensor de força e torque com feedback de temperatura e calibração ajustável
- Em alguns casos o ros2_control tem definido já componentes semânticos para sensores/tipos de leituras comuns
 - IMUSensor
 - ForceTorqueSensor
 - PoseSensor
 - GPSSensor

```
<ros2_control name="RRBotForceTorqueSensor2D" type="sensor">
  <hardware>
    <plugin>ros2_control_demo_hardware/ForceTorqueSensor2DHardware</plugin>
    <param name="example_param_read_for_sec">0.43</param>
  </hardware>
  <sensor name="tcp_fts_sensor">
    <state_interface name="fx" />
    <state_interface name="tz" />
    <param name="frame_id">kuka_tcp</param>
    <param name="fx_range">100</param>
    <param name="tz_range">100</param>
  </sensor>
  <sensor name="temp_feedback">
    <state_interface name="temperature" />
  </sensor>
  <gpio name="calibration">
    <command_interface name="calibration_matrix_nr" />
    <state_interface name="calibration_matrix_nr" />
  </gpio>
</ros2_control>
```


CONFIGURANDO E LANÇANDO CONTROLADORES

- Controladores (e o Controller Manager) devem ser configurados em arquivos .YAML para definir como eles serão rodados
 - Permite alterarmos ganhos, taxas de atualização, etc.
 - Definimos também o tipo de controladores que serão carregados pelo controller manager
 - No exemplo à direita, ele carrega três controladores:
 - joint_state_broadcaster
 - forward_position_controller
 - joint_trajectory_position_controller
 - Arquivos de configuração devem ser carregado pelos arquivos de launch ao executar o programa

```
# Arquivo de configuração do rrbot, exemplo 1
controller_manager:
  ros__parameters:
    update_rate: 10 # Hz

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    forward_position_controller:
      type: forward_command_controller/ForwardCommandController

    joint_trajectory_position_controller:
      type: joint_trajectory_controller/JointTrajectoryController

forward_position_controller:
  ros__parameters:
    joints:
      - joint1
      - joint2
    interface_name: position

joint_trajectory_position_controller:
  ros__parameters:
    joints:
      - joint1
      - joint2

command_interfaces:
  - position

state_interfaces:
  - position

state_publish_rate: 200.0 # Defaults to 50
action_monitor_rate: 20.0 # Defaults to 20

allow_partial_joints_goal: false # Defaults to false
open_loop_control: true
allow_integration_in_goal_trajectories: true
constraints:
  stopped_velocity_tolerance: 0.01 # Defaults to 0.01
  goal_time: 0.0 # Defaults to 0.0 (start immediately)
```

ESCREVENDO UMA INTERFACE COM ROS2 CONTROL

217

- Se formos criar uma `HardwareInterface`, precisamos implementá-la como uma subclasse adequada de `hardware_interface::($InterfaceType)Interface`
- Classe deve implementar os métodos solicitados de `LifecycleNodeInterface` (**opcionais em azul**) e sobrescrever os **métodos requisitados pela sua `HardwareInterface`** (e.g. `SensorInterface` precisa de `read()`, mas não `write()`, e `export_state_interfaces()`, mas não `export_command_interfaces()`)

LifecycleNodeInterface	(\$InterfaceType)Interface
<code>on_configure()</code>	<code>on_init()</code>
<code>on_cleanup()</code>	<code>read()</code>
<code>on_shutdown()</code>	<code>write()</code>
<code>on_activate()</code>	<code>export_state_interfaces()</code>
<code>on_deactivate()</code>	<code>export_command_interfaces()</code>
<code>on_error()</code>	<code>prepare_command_mode_switch()</code>
	<code>perform_command_mode_switch()</code>

Guia:

https://control.ros.org/jazzy/doc/ros2_control/hardware_interface/doc/writing_new_hardware_component.html

Exemplo no `ros2_control_demos`:
plugin "RobotController" do ex 7

No final da definição da classe, source incluir o macro

```
PLUGINLIB_EXPORT_CLASS(namespace_do_hardware::nome_do_hardware, tipo_de_hardwareinterface)
```

ESCREVENDO UM CONTROLLER COM ROS2_CONTROL

218

- Se formos criar um componente Controller, ele deve ser escrito em um arquivo header (.hpp) e source (.cpp) de C++
- No header, incluir "controller_interface/controller_interface.hpp"
 - O controlador estende controller_interface::ControllerInterface
 - Implementa os seguintes métodos públicos:

São executados
em tempo real

Opcionais

```
on_init()  
command_interface_configuration()  
state_interface_configuration()  
on_configure()  
on_activate()  
on_deactivate()  
update()  
on_cleanup()  
on_shutdown()  
on_error()
```

Guia:

https://control.ros.org/jazzy/doc/ros2_controllers/doc/writing_new_controller.html

Exemplo no pacote

ros2_control_demos: plugin
"RobotController" do exemplo 7

No final da definição da classe, source incluir o macro

```
PLUGINLIB_EXPORT_CLASS(namespace_do_controlador::nome_do_controlador, interface_do_controlador)
```

EXPORTANDO O PLUGIN

- O plugin que usa a pluginlib (caso de HardwareInterface e Controllers no ros2_control) deve ter um arquivo <nome_do_plugin>.xml no diretório base do pacote para ser usado e descoberto pela pluginlib.
- Deve conter nome de classe, tipo, e tipo base do plugin que será exportado.

```
<!-- q2d_hardware.xml -->
<!-- Código de autoria do Prof. Walter Fetter Lages -->
<library path="q2d_hardware">
  <class name="q2d_hardware/Q2dSystemHardware"
        type="q2d_hardware::Q2dSystemHardware"
        base_class_type="hardware_interface::SystemInterface">
    <description>
      Quanser 2DSFJ system hardware interface.
      It uses effort command interfaces and
      position and velocity state interfaces.
    </description>
  </class>
</library>
```

Exemplo para plugin q2d_hardware, do tipo base
hardware_interface::SystemInterface

```
<!-- pid_controller.xml -->
<library path="pid_controller">
  <class name="pid_controller/PidController"
        type="pid_controller::PidController"
        base_class_type="controller_interface::ChainableControllerInterface">
    <description>
      PidController ros2_control controller.
    </description>
  </class>
</library>
```

Exemplo para plugin pid_controller, do tipo base
controller_interface::ChainableControllerInterface

DEPENDÊNCIAS EM CMAKELISTS.TXT E PACKAGE.XML

- Devemos adicionar as dependências necessárias para o plugin tanto em package.xml e CMakeLists.txt

```
<!-- Dependências a adicionar em package.xml -->
<depend>hardware_interface</depend>
<depend>pluginlib</depend>
<depend>rclcpp</depend>
<depend>rclcpp_lifecycle</depend>

<!-- Dependências para teste: verifica se plugin é descoberto -->
<test_depend>ament_cmake_gmock</test_depend>
<test_depend>hardware_interface_testing</test_depend>

<!-- Abaixo: caso seja um plugin de controlador-->
<test_depend>controller_manager</test_depend>
<test_depend>ros2_control_test_assets</test_depend>
```

Linhas que devem estar presentes em package.xml
(Nota: se for controlador, incluir também dependência em controller_interface)

```
## Dependências obrigatórias, eventualmente outras também
find_package(rclcpp REQUIRED)
find_package(rclcpp_lifecycle REQUIRED)
find_package(hardware_interface REQUIRED)
find_package(controller_interface REQUIRED) #SÓ SE FOR UM CONTROLADOR!
find_package(pluginlib REQUIRED)

## Instruções de compilação para uma biblioteca compartilhada
add_library(nome_da_biblioteca SHARED src/nome_da_biblioteca.cpp)
target_include_directories(nome_da_biblioteca PRIVATE
  ${CMAKE_CURRENT_SOURCE_DIR}/include
  ${CMAKE_CURRENT_BINARY_DIR}/include)

## Dependências usadas pela biblioteca: incluir pelo menos as acima
ament_target_dependencies(nome_da_biblioteca ...)
target_include_directories(nome_da_biblioteca PUBLIC ...) #Dependências e links conforme requerido
target_link_libraries(nome_da_biblioteca PUBLIC ...)

## Exporta para um arquivo de descrição pluginlib: escolher conforme tipo de plugin
pluginlib_export_plugin_description_file(hardware_interface nome_do_hardware.xml) #se hardware
pluginlib_export_plugin_description_file(controller_interface nome_do_controlador.xml) #se controlador

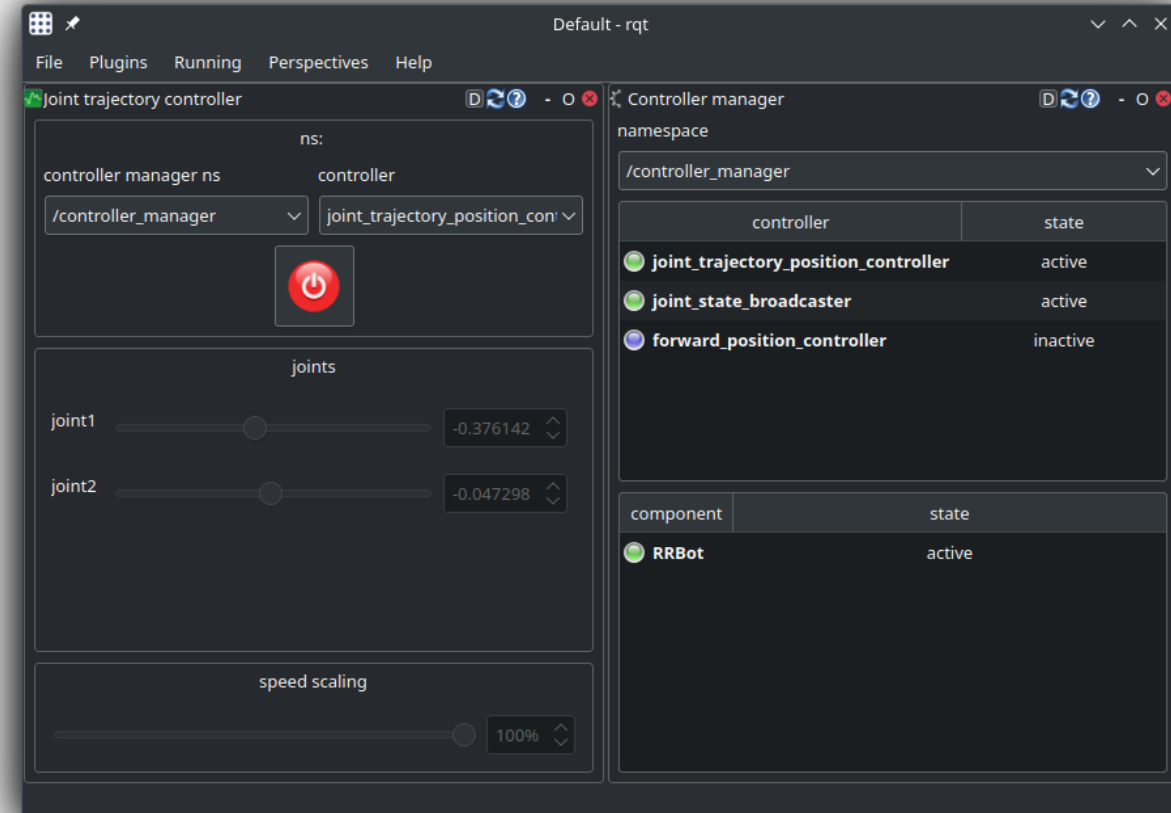
## Diretivas de instalação apropriadas
install(DIRECTORY include DESTINATION include)
install(TARGETS nome_da_biblioteca EXPORT export_${PROJECT_NAME} ...)

## Exports no fim do arquivo
ament_export_include_directories(include) # e eventualmente outras dependências
ament_export_libraries(nome_da_biblioteca)
ament_export_targets(export_${PROJECT_NAME})
ament_package()
```

Linhas que devem estar presentes em CMakeLists.txt

PLUGINS DO ROS2_CONTROL

- Plugin do rqt para interfacear com `ros2_control` de maneira gráfica
- Podem ser instalados via cli:
 - `sudo apt install ros-${ROS_DISTRO}-rqt_controller_manager`
- Depois de instalados, devem ser descobertos pelo rqt:
 - `rqt --force-discover --standalone rqt_controller_manager`

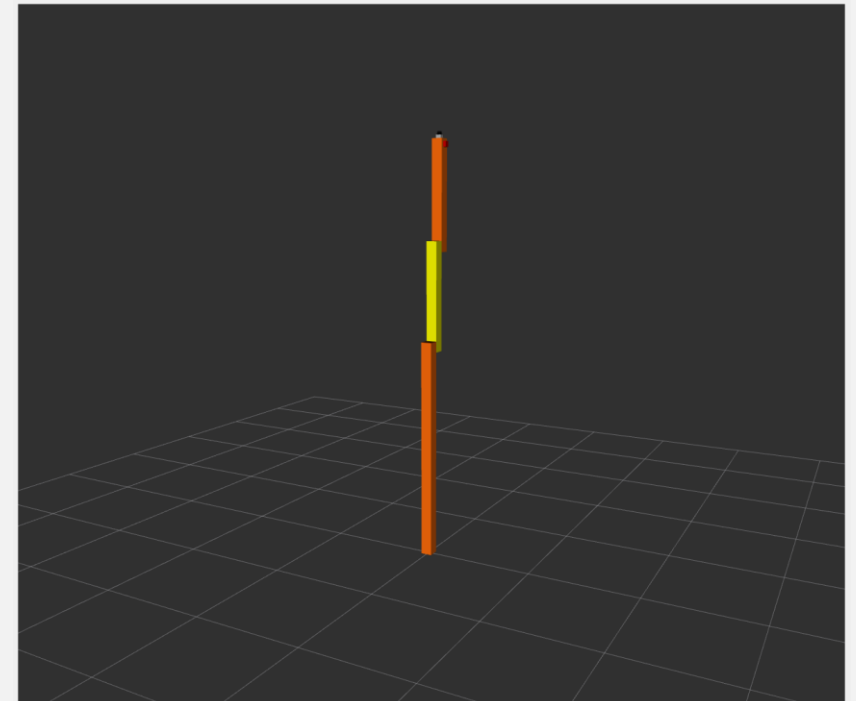


Plugins do `ros2_control` no rqt.
À esquerda: joint trajectory controller.
À direita: controller manager.

DEMONSTRAÇÃO: CONTROLE DO RRBOT

Exemplo da documentação do `ros2_control`

- Controle de um robô planar com 2 juntas revolutas
- Plugin de hardware é implementado com só 1 interface
- Instruções para acompanhar a demonstração:
https://control.ros.org/jazzy/doc/ros2_control_demos/example_1/doc/userdoc.html
- É necessário ter [instalado](#) o repositório com os pacotes do `ros2_control_demos`
- Exemplo 7 também é completo, para quem tiver interesse



Vista do rrbot no RViz2.